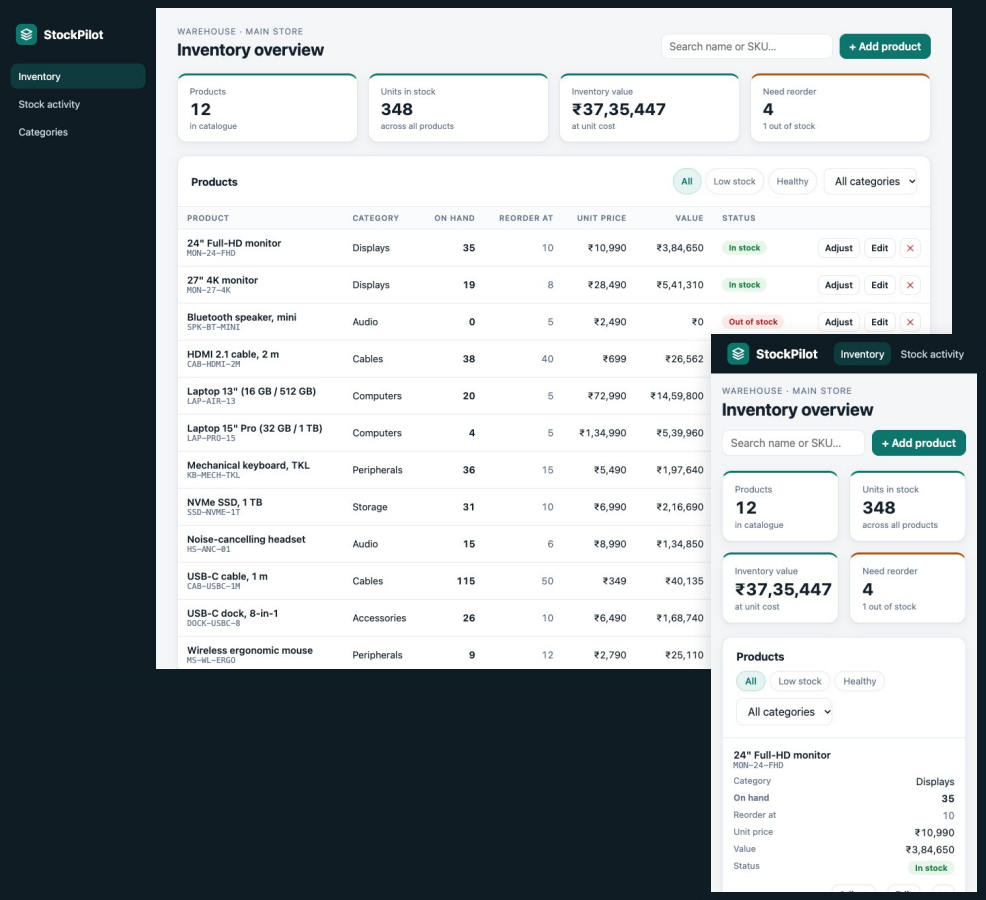
SESSION 21 · DEVOPS FINAL CAPSTONE

StockPilot

Inventory management, from a laptop to a monitored Kubernetes cluster on AWS



Lakshya Mewara

Enrollment 24BCS10290

What StockPilot does

Product catalogue

SKU, category, unit price and a reorder level for every product

Audited stock movements

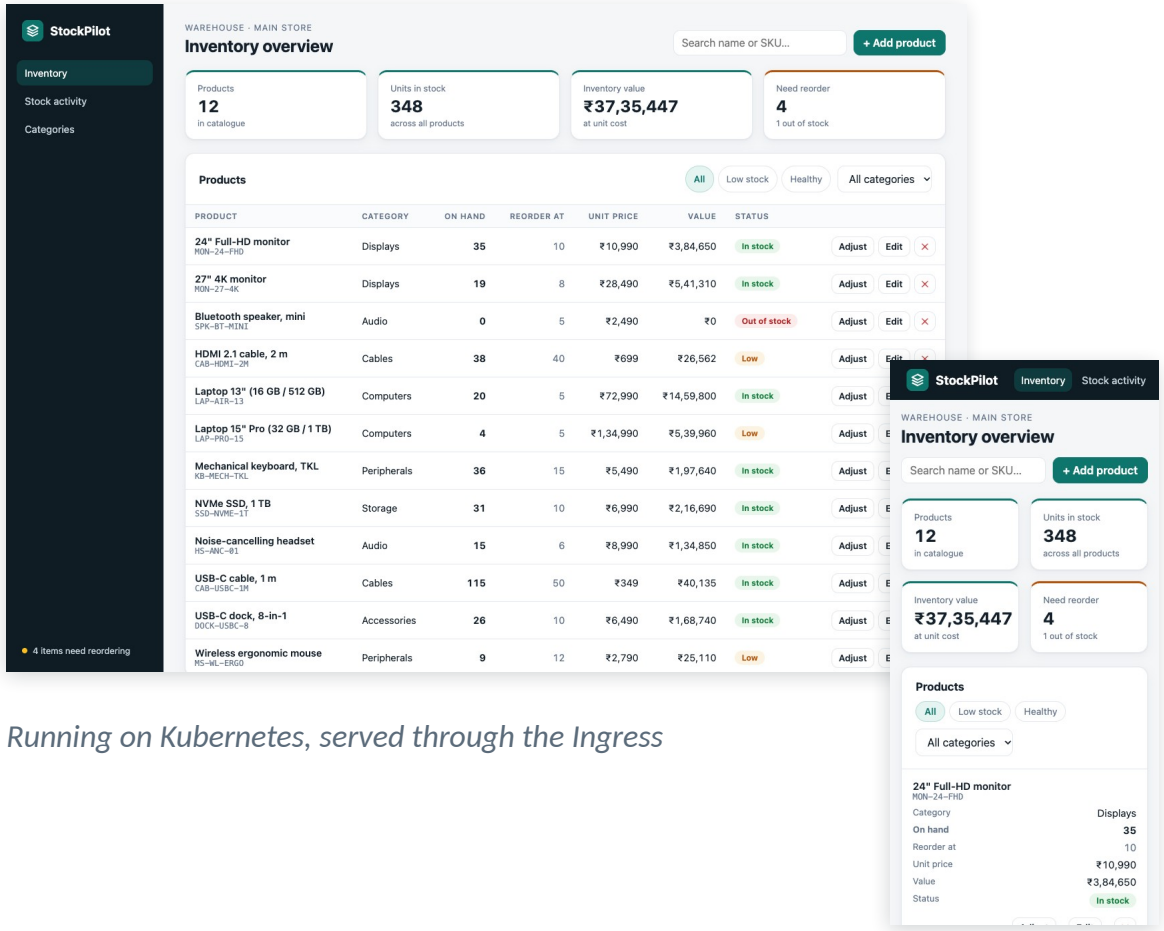
Every stock in and out is recorded; stock can never go negative

Reorder alerts

Low and out-of-stock items flagged in the UI and in Prometheus

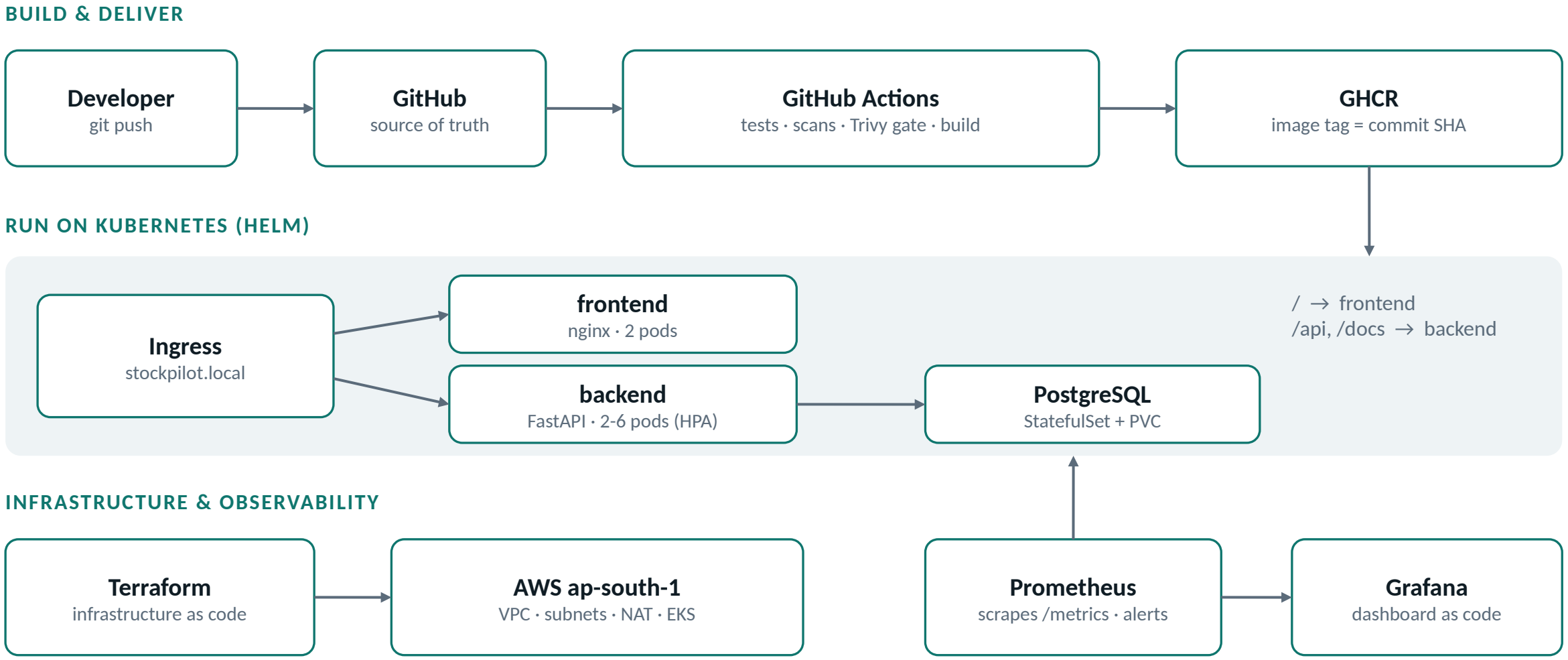
Responsive dashboard

KPIs, search, filters and a live activity feed on desktop and phone



Running on Kubernetes, served through the Ingress

Architecture at a glance



Tech stack



React + Vite

Responsive dashboard



FastAPI

REST API, 10 endpoints



PostgreSQL

Data + Alembic migrations



pytest

20 automated tests



Docker

2 images + Compose



GitHub Actions

9-job CI/CD pipeline



Trivy

Image security gate



Terraform

AWS VPC + EKS



Kubernetes

k3s locally, EKS on AWS



Helm

One chart, 3 value sets



Prometheus

Metrics and alert rules



Grafana

Dashboard shipped as code

Application: API and data

10

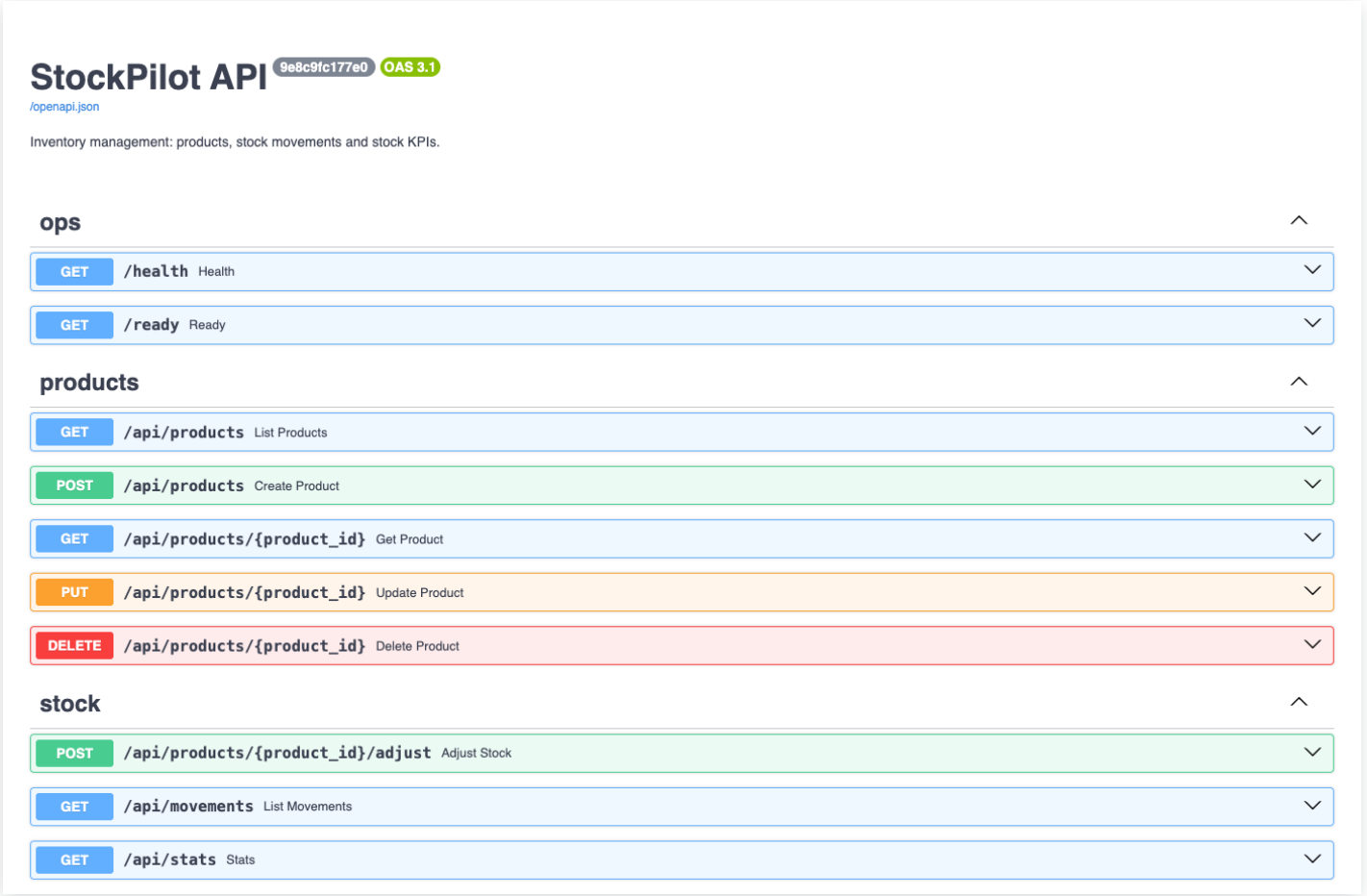
REST endpoints - GET, POST, PUT, DELETE, plus stock adjust

2

Alembic migrations - products and the stock_movements audit trail

1

Postgres advisory lock - replicas run migrations one at a time



Testing: the first quality gate

20

pytest tests, all passing

89%

code coverage

11

routes covered: every endpoint + /metrics

Throwaway test database

A fresh SQLite DB built by the real Alembic migrations - production is never touched

Real PostgreSQL in CI

Migrations go upgrade → downgrade → upgrade on Postgres 18 to prove they reverse

Fail fast

A failing test stops the pipeline before any image is built or pushed

Docker: two images, one command

Backend image

python:3.12-slim, dependencies cached in their own layer.
On start: wait for DB → alembic upgrade → uvicorn.

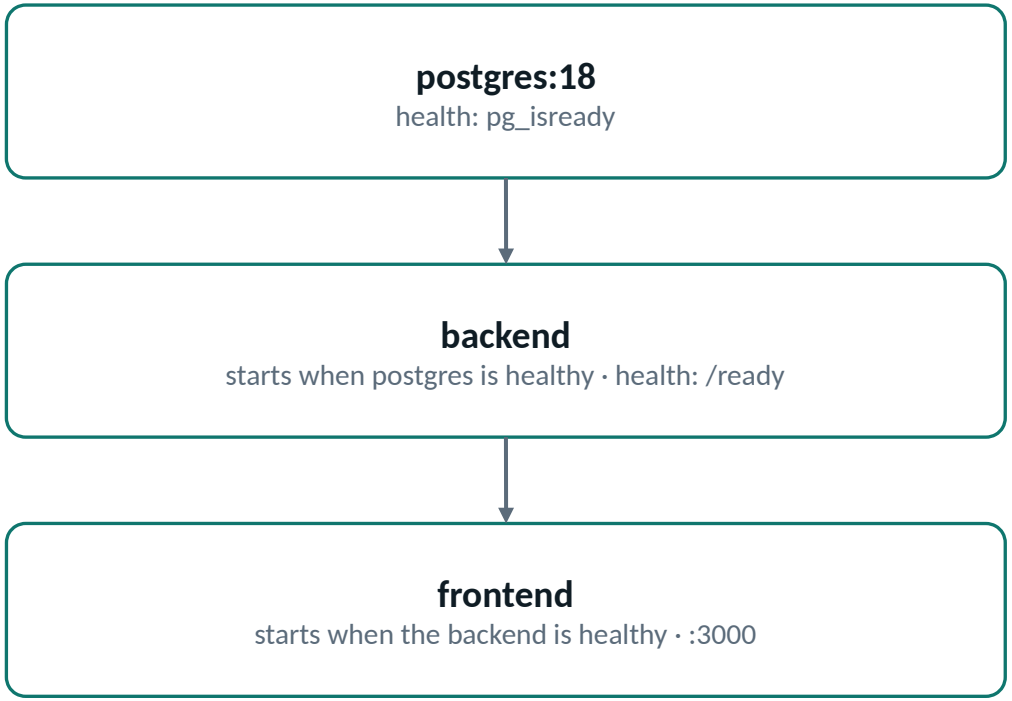
Runs as UID 10001

Frontend image

Multi-stage: Node 22 builds the React app, then unprivileged nginx serves only the static files.

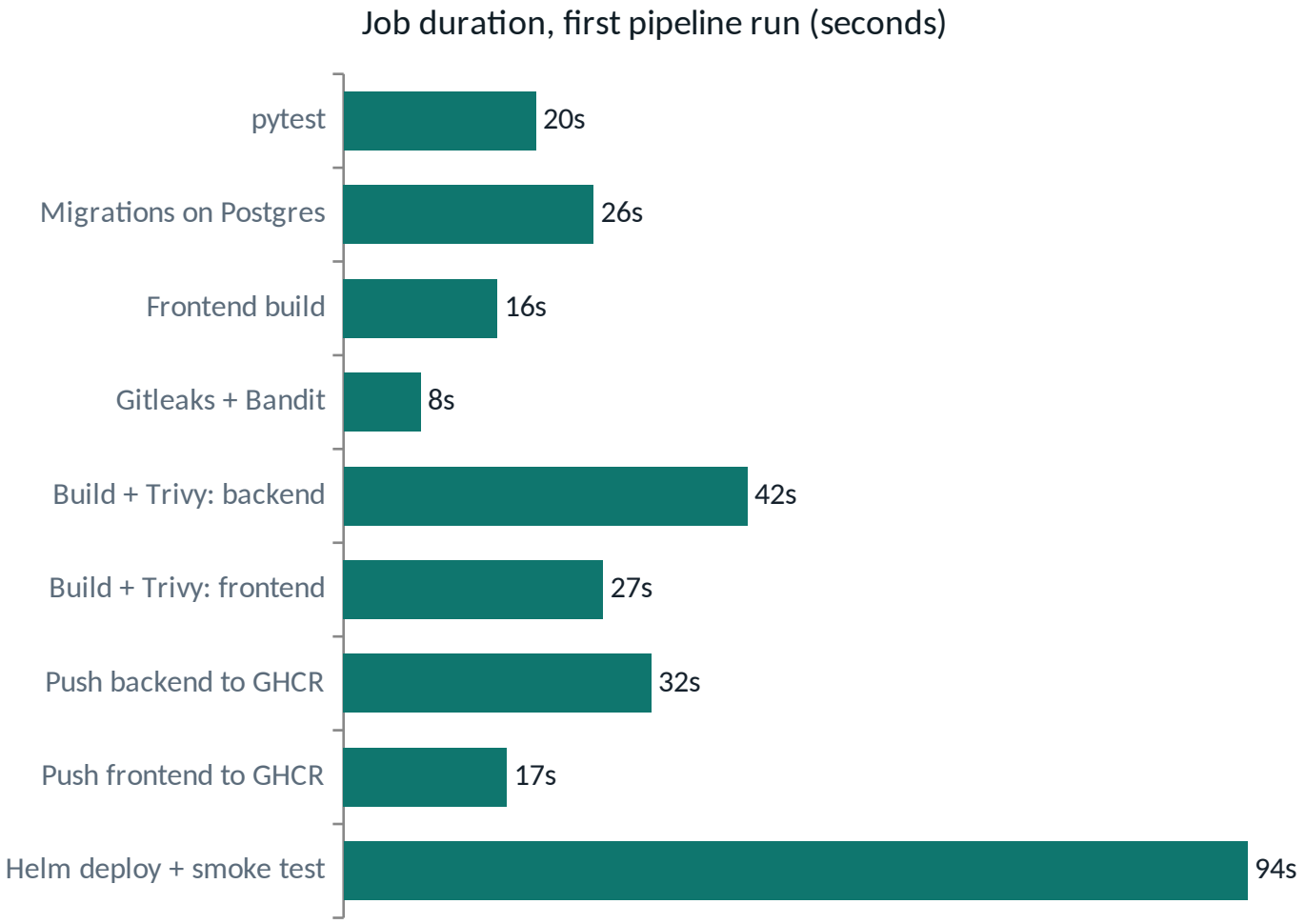
Runs as UID 101

```
docker compose up --build
```



3 / 3 services healthy

CI/CD: nine jobs on every push



4 / 4

pipeline runs green

SHA

every image is tagged with its full commit SHA, never 'latest'

0

rebuilds after scanning: the exact scanned image is pushed

DevSecOps: a gate that really blocks



Scan fails

Trivy finds CVE-2026-4775 (HIGH) in libtiff inside the nginx base image. A fix exists, so the gate exits 1 and nothing is pushed.

Fix

The runtime stage runs apk upgrade, pulling the patched libtiff 4.7.2-r0.

Pass

0 fixable HIGH or CRITICAL vulnerabilities in either image - the gate passes and the images are pushed.

Trivy
container images

Gitleaks
no secrets in Git

Bandit
Python SAST

npm audit
frontend dependencies

At runtime: non-root, read-only root filesystem, all Linux capabilities dropped

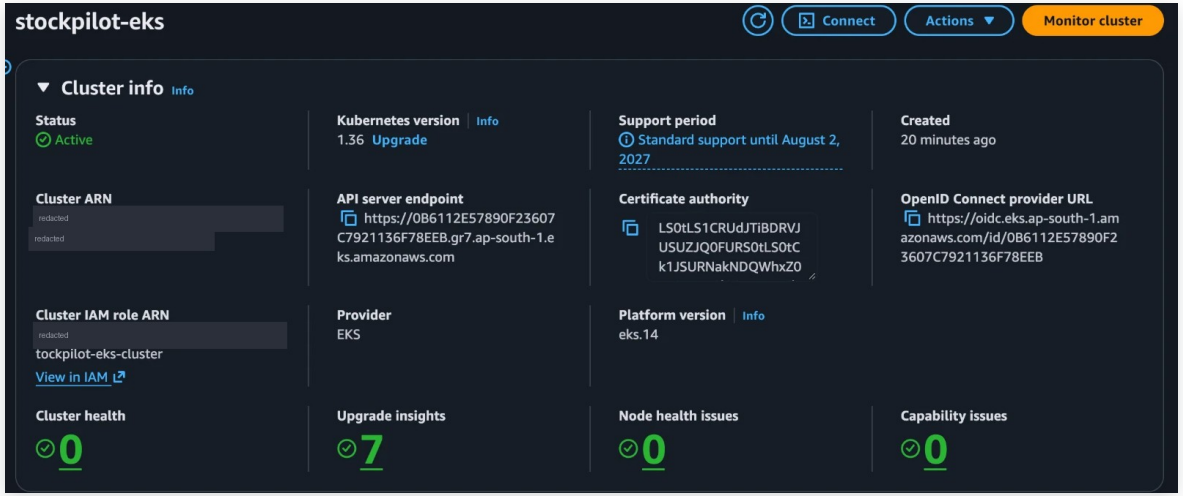
Terraform: AWS VPC + EKS in Mumbai

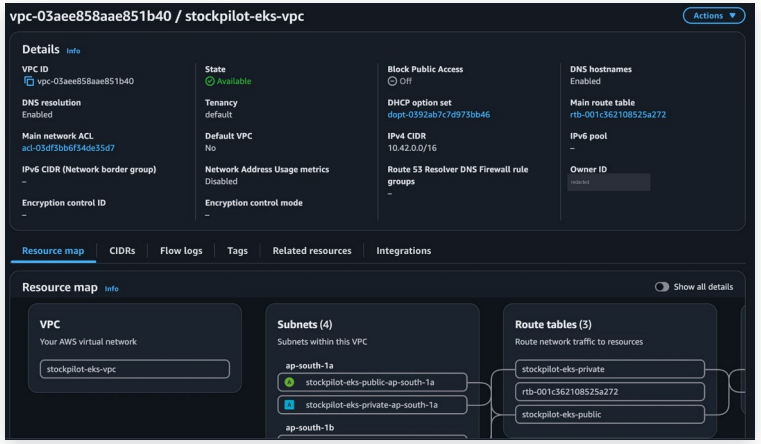
22
resources, one plan

11½ min
terraform apply

0
left after destroy

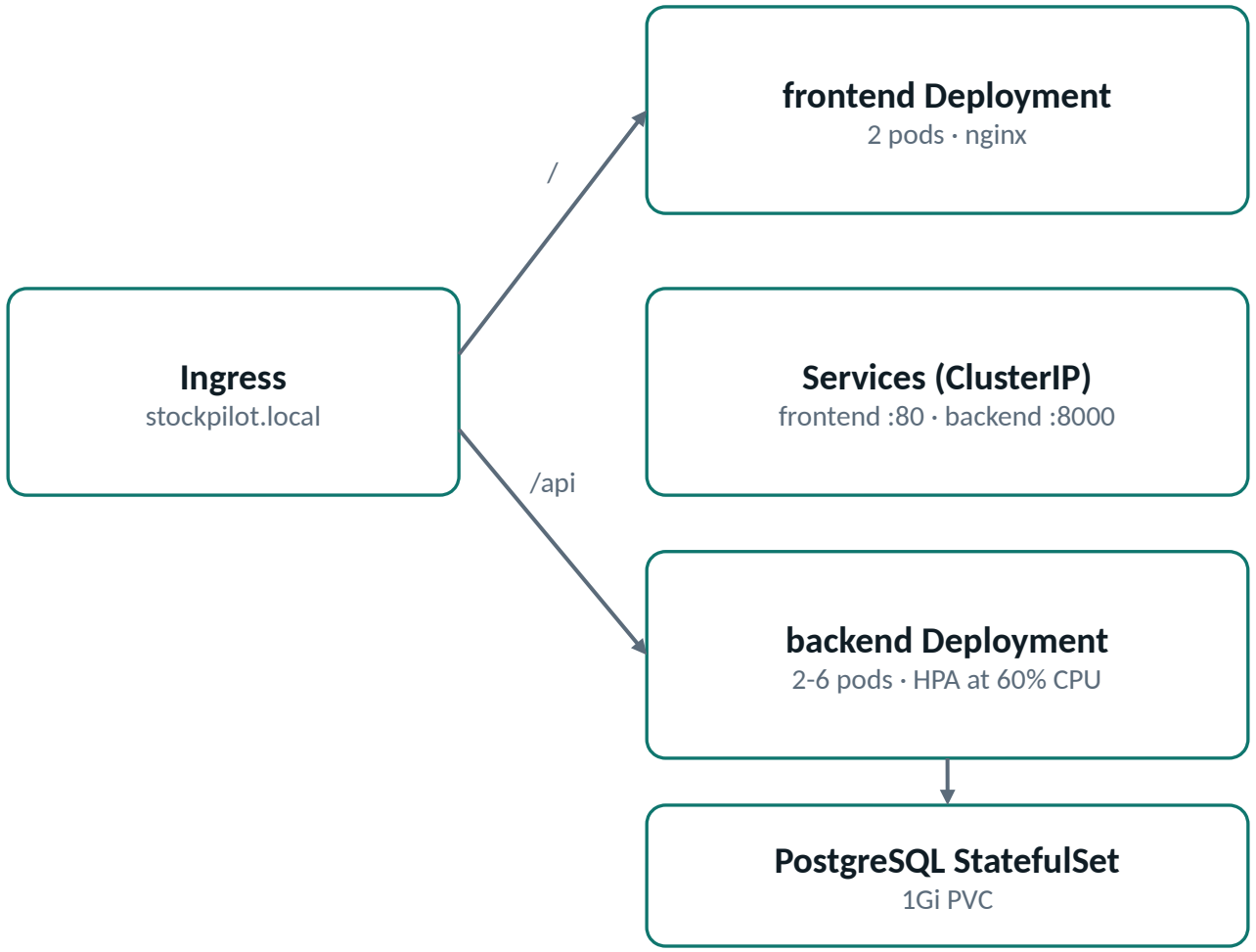
- VPC 10.42.0.0/16 across 2 availability zones
- 2 public subnets (load balancers) + 2 private subnets (nodes)
- Internet gateway, NAT gateway, route tables
- IAM roles for the cluster and the nodes
- EKS 1.36 + managed node group: 2 × t3.medium
- API endpoint restricted to one IP; no keys in code





AWS Console: the EKS cluster Active on 1.36, and the VPC with its 4 subnets. Account details redacted.

Kubernetes + Helm

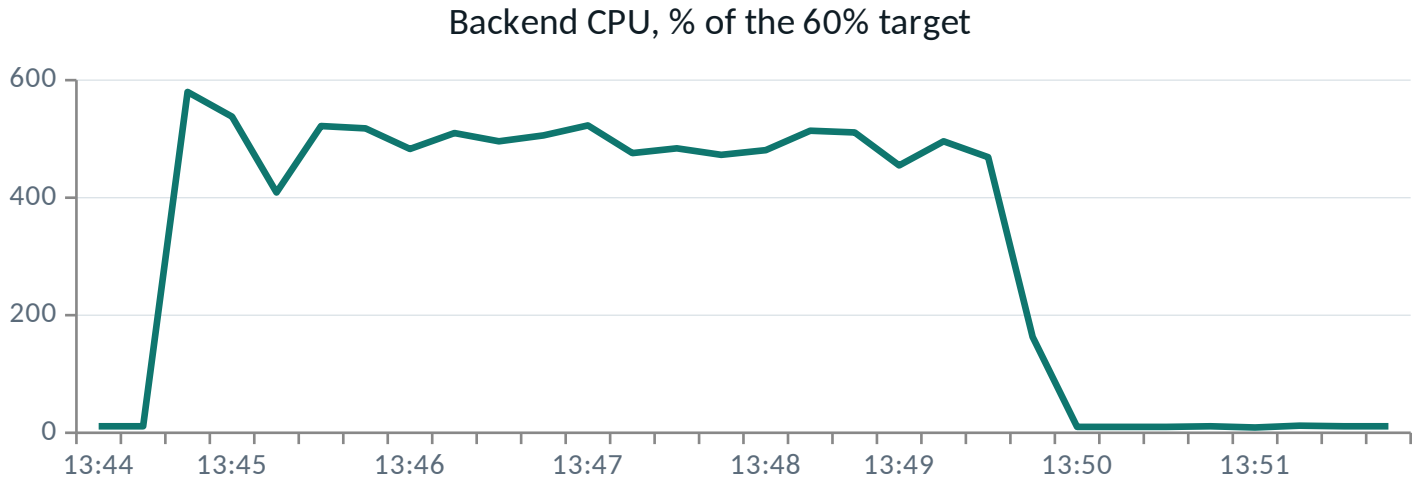


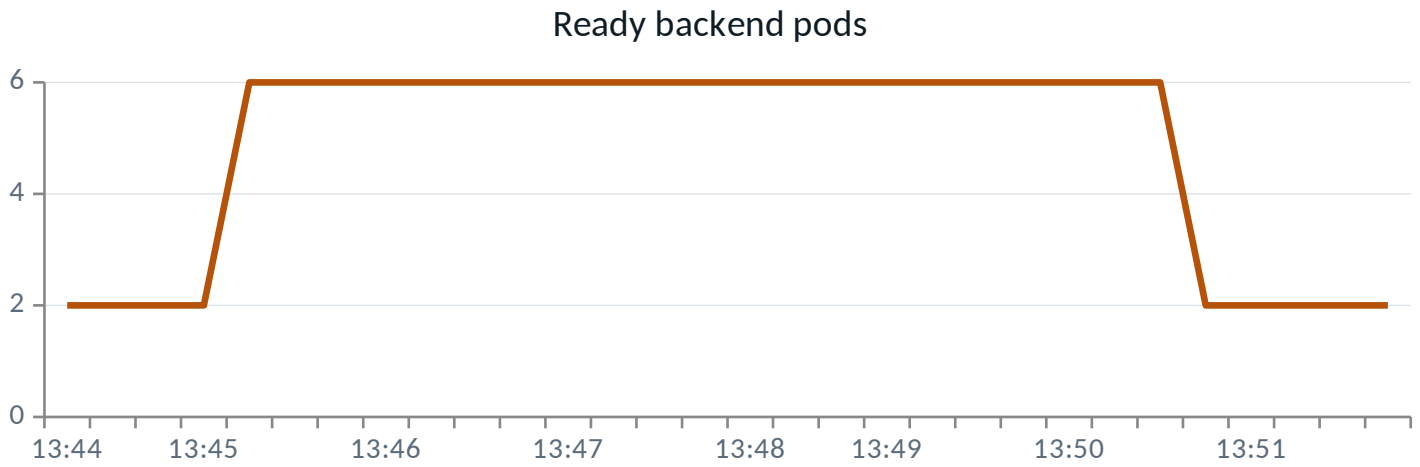
One Helm chart deploys

- 2 Deployments
- 3 Services
- StatefulSet + PVC
- Ingress
- HorizontalPodAutoscaler
- 2 PodDisruptionBudgets
- ConfigMap + Secret
- ServiceMonitor
- PrometheusRule
- Grafana dashboard

helm lint passes with dev, default and prod values

Autoscaling under real load





580%

of the CPU target when load started

2 → 6

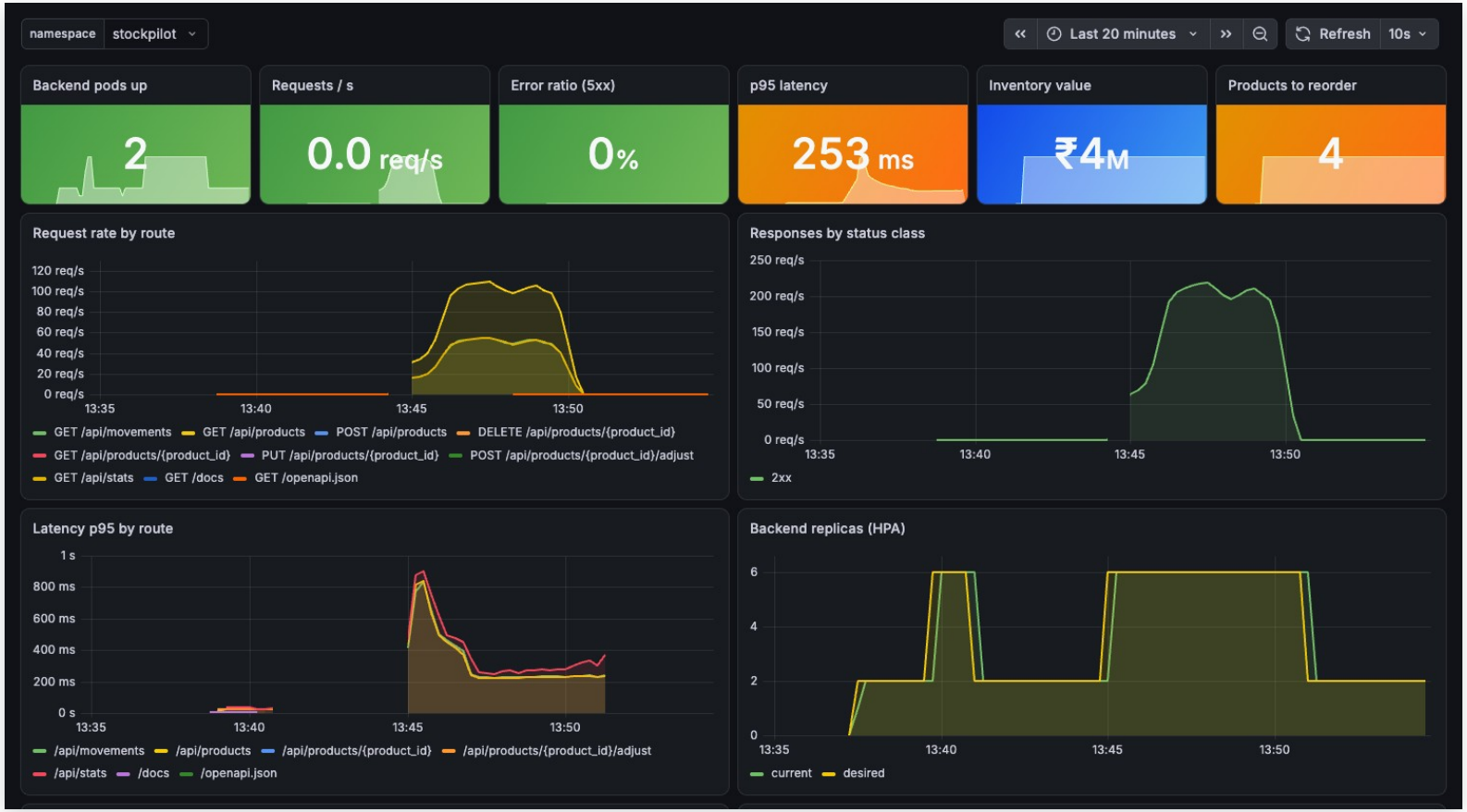
pods, all Ready within 30 s

≈ 75 s

after the load stopped, back to 2 pods

At 6 pods the single node hit 96% CPU: the HPA adds pods, not machines. On EKS a cluster autoscaler adds nodes.

Observability: Prometheus + Grafana



StockPilot dashboard after the load test: request rate, status codes, p95 latency, replicas, CPU and stock

6 / 6 targets UP

a ServiceMonitor scrapes every backend pod's /metrics

5 alert rules

backend down, 5xx ratio, p95 latency, HPA at max, and a business alert for products to reorder

Dashboard as code

ships in the Helm chart; Grafana loads it automatically

Troubleshooting lab

SYMPTOM	INVESTIGATE	ROOT CAUSE	FIX
Pods stuck in ImagePullBackOff	kubectl describe pod: image reference not found	Image tag typo: 1.31-alpinee	kubectl set image → rollout succeeds
Pods Ready, but requests to the Service fail	EndpointSlice is empty; compare the selector with --show-labels	Service selects app=catalog-wbe, matching no pod	Patch the selector → 2 endpoints, HTTP 200

What I learned



Deploying finds what tests miss

Migrations could not import the app inside the container - only docker compose caught it



Test the gates too

Trivy really blocked a build on a fixable HIGH CVE until the image was patched



Metrics need care

Default 0.1 s histogram buckets made p95 meaningless; millisecond buckets fixed it



Check limits before apply

A shared account had 5 of 5 Elastic IPs used, so the NAT gateway became optional



Autoscaling has a ceiling

The HPA adds pods, not nodes - at 6 pods the node was at 96% CPU

LIVE DEMO

Thank you

Commit a change → watch the pipeline → new image in GHCR → helm upgrade → refresh the app



Project repository

github.com/LAKSHYAMEWARA0025/devops-heros/tree/main/session21-capstone

README: setup, evidence and the full demo script

Lakshya Mewara

Enrollment 24BCS10290